

TEST STRATEGY DOCUMENT

ERP Web Application Testing

Document Type	Test Strategy	Document Version	1.0
Project	ERP Web Application	Methodology	Agile – Scrum
Testing Approach	Object, UI, Functional & Integration	Defect Tracking	JIRA
Manual Testing	MS Excel documentation	Automation	Selenium WebDriver, Java, TestNG, Maven
Prepared By	Sivamurugan G	Date	[DATE]

1. Introduction

The purpose of this Test Strategy is to define the overall testing approach, scope, objectives, resources, testing techniques, tools, and quality criteria for the ERP Web Application.

The application consists of multiple business modules that support core organizational processes such as finance, sales, purchasing, inventory, production, human resources, logistics, and service management.

The testing process is designed to identify functional defects, usability issues, integration problems, and inconsistencies between business requirements and application behavior.

2. Test Objectives

- Verify that the application functions according to defined business requirements.
- Validate critical business workflows across ERP modules.
- Identify defects at an early stage of the development lifecycle.
- Verify the usability and consistency of the application UI.
- Validate data flow between integrated modules.
- Ensure that changes do not negatively affect existing functionality.
- Maintain traceability between requirements, test scenarios, test cases, and defects.
- Provide reliable test results to support release decisions.
- Improve overall application quality and release confidence.

3. Scope of Testing

Module	Submodules
Finance	General Ledger; Accounts Payable; Accounts Receivable; Fixed Assets
Sales	Customer Management; Quotation Management; Sales Order Management; Invoicing
Production	Production Planning; Work Orders; Bill of Materials; Production Tracking
Human Resources	Employee Management; Attendance Management; Leave Management; Payroll
Purchase	Vendor Management; Purchase Requisition; Purchase Order; Purchase Invoice
Inventory	Item Management; Stock Management; Warehouse Management; Stock Movement
Logistics	Shipment Management; Delivery Management; Transportation Management; Tracking

Service	Service Request; Ticket Management; Service Assignment; Service Status Tracking
----------------	---

4.1 Object Testing

Object testing validates individual application objects and their behavior, including buttons, input fields, dropdowns, checkboxes, tables, links, pop-up windows, and navigation elements.

4.2 UI Testing

UI testing validates visual and interactive behavior, including page layout, field alignment, labels, buttons, navigation, fonts, error messages, input controls, and consistency between screens.

4.3 Functional Testing

Functional testing validates whether application functionality works according to business requirements, including creating, updating, deleting, searching, invoicing, payments, purchasing, inventory, and employee management.

4.4 Integration Testing

Integration testing validates data flow and interaction between related ERP modules. Examples include Sales Order → Inventory → Delivery → Invoice → Finance, and Purchase Order → Inventory → Purchase Invoice → Accounts Payable.

5. Testing Approach

The testing approach follows a structured process from requirement analysis through test closure.

Requirement Analysis → Test Scenario Identification → Test Case Design → Test Data Preparation → Test Execution → Defect Reporting → Defect Retesting → Regression Testing → Test Closure.

6. Test Case Design Strategy

- Business requirements and functional specifications.
- User workflows and end-to-end business processes.
- Positive and negative scenarios.
- Boundary conditions and validation rules.
- Integration points and error handling requirements.
- Test cases will capture Test Case ID, Module, Submodule, Test Scenario, Preconditions, Test Data, Test Steps, Expected Result, Actual Result, Status, Defect ID, and Remarks.

7. Test Data Strategy

- Positive data: valid values used to verify successful transactions.
- Negative data: invalid or unexpected values used to validate error handling.
- Boundary data: values at minimum, maximum, and limit conditions.
- Integration data: data required to validate information flow between ERP modules.

Example workflow: Customer → Quotation → Sales Order → Delivery → Invoice → Payment.

8. Regression Testing Strategy

- Regression testing will be performed after new functionality is implemented.
- Existing functionality will be revalidated after modifications and defect fixes.
- Relevant regression suites will be executed after integration changes and major application releases.
- The objective is to ensure that existing functionality continues to work after changes are introduced.

9. Smoke Testing Strategy

Smoke testing will be performed on a new build to determine whether the application is stable enough for detailed testing.

Critical areas include application login, navigation, main module access, record creation, record search, core transaction processing, and application connectivity.

If critical smoke test cases fail, detailed testing may be postponed until the build is stable.

10. Sanity Testing Strategy

Sanity testing will be performed after specific changes or defect fixes to verify that the reported defect has been fixed, related functionality continues to work, and the modified functionality is stable enough for further testing.

11. Defect Management Strategy

- Defects identified during testing will be documented and tracked using JIRA.
- Each defect should contain Defect ID, Summary, Description, Module, Environment, Steps to Reproduce, Expected Result, Actual Result, Severity, Priority, Evidence/Screenshot, Assignee, and Status.
- Defect lifecycle: New → Assigned → Open → Fixed → Retest → Verified → Closed.
- If a defect is not fixed correctly: Retest → Failed → Reopened.

12. Severity and Priority

Severity	Description
High	Major functionality is unavailable or a critical business workflow is blocked.
Medium	Functionality is affected but a workaround or alternative path may exist.
Low	Minor functional, UI, validation, or cosmetic issue with limited impact.

Priority will determine how quickly a defect should be addressed based on business impact and release requirements.

13. Requirements Traceability

A Requirement Traceability Matrix (RTM) will be maintained to establish traceability between requirements and testing activities.

The RTM will connect Requirement → Test Scenario → Test Condition → Test Case → Execution Result → Defect, helping ensure that important requirements are adequately covered.

14. Automation Testing Strategy

Automation testing will be implemented for suitable and repeatable test scenarios.

Technology stack: Java, Selenium WebDriver, TestNG, Maven, Page Object Model, and Git/GitHub.

Suitable scenarios include repetitive test cases, regression test cases, stable application workflows, data-driven scenarios, cross-browser validation, and frequently executed functional scenarios.

Manual testing will continue to be used for exploratory testing, usability validation, new functionality, and scenarios where automation provides limited value.

15. Test Execution Strategy

- Build Verification: perform smoke testing to confirm build stability.
- Functional Execution: execute module-level functional test cases.
- Integration Execution: validate workflows involving multiple modules.

- Regression Execution: execute relevant regression suites after changes and defect fixes.
- Automation Execution: execute automated regression scenarios where applicable.
- Defect Retesting: retest resolved defects and verify their fixes.

16. Entry Criteria

- Requirements are available and sufficiently understood.
- Testable functionality has been deployed.
- Required test environment is available.
- Test data is prepared.
- Test scenarios and test cases are ready.
- Required testing tools are accessible.
- Critical blocking issues preventing testing have been addressed.

17. Exit Criteria

- Planned test cases have been executed.
- Critical business workflows have been validated.
- High-severity defects have been resolved or formally accepted.
- Required regression testing has been completed.
- Defect retesting has been completed.
- Test results have been documented.
- RTM coverage has been reviewed.
- Remaining known issues have been communicated to stakeholders.

18. Test Deliverables

- Test Strategy
- Test Scenarios
- Test Cases
- Test Conditions
- Test Data
- Requirement Traceability Matrix
- Defect Reports
- Test Execution Results
- Regression Test Results
- Automation Test Scripts
- Test Summary Report

19. Roles and Responsibilities

Role	Responsibility
QA Tester	Requirement analysis, test design, execution, defect reporting and retesting.

Automation Tester	Development and maintenance of automated test scripts.
Developer	Defect analysis and resolution.
Business Analyst	Requirement clarification and validation.
Project Manager	Project coordination and release planning.
QA Lead	Test planning, review, reporting and quality oversight.

20. Risk and Mitigation

Risk	Impact	Mitigation
Requirement changes	High	Review and update test cases and RTM.
Unstable builds	High	Perform smoke testing before detailed execution.
Environment issues	High	Validate environment before execution.
Incomplete test data	Medium	Prepare test data in advance.
Defect fixing delays	Medium	Prioritize defects based on severity and business impact.
Integration issues	High	Execute end-to-end integration scenarios.
Automation script instability	Medium	Maintain reusable and modular automation components.

21. Test Environment

- Application: ERP Web Application
- Platform: Web Browser
- Operating System: Windows
- Browsers: Chrome / Edge
- Database: Application-specific database
- Automation: Selenium WebDriver
- Programming Language: Java
- Framework: TestNG
- Build Tool: Maven
- Defect Tracking: JIRA
- Documentation: MS Excel
- Version Control: Git / GitHub

22. Metrics and Reporting

- Total test cases
- Executed test cases
- Passed, failed, and blocked test cases
- Test execution percentage
- Defect count
- Defects by severity
- Defects by module
- Defect closure percentage

- Requirement coverage
- Regression pass percentage

These metrics help stakeholders understand the current quality and testing status of the application.

23. Overall Testing Workflow

Requirements → Requirement Analysis → Test Planning → Test Scenario Design → Test Case Design → Test Data Preparation → Smoke Testing → Functional Testing → UI Testing → Integration Testing → Defect Reporting → Defect Retesting → Regression Testing → Automation Execution → RTM Update → Test Summary → Test Closure.

24. Conclusion

This Test Strategy establishes a structured and risk-based approach for validating the ERP Web Application. Testing will focus on functional correctness, user interface behavior, business workflow validation, module integration, regression coverage, and defect management.

The combination of manual testing practices with Selenium-based automation will provide broader coverage, improve regression efficiency, and support consistent validation of critical application workflows.

The overall objective is to identify defects early, maintain requirement traceability, validate business-critical processes, and provide stakeholders with reliable information regarding application quality and release readiness.